Федеральное государственное автономное образовательное
учреждение высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
"ВЫСШАЯ ШКОЛА ЭКОНОМИКИ"

Факультет компьютерных наук
Департамент программной инженерии

ТРЕБОВАНИЯ К ВЫПОЛНЕНИЮ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ №3 ПО ДИСЦИПЛИНЕ «АРХИТЕКТУРА КОМПЬЮТЕРА И ОПЕРАЦИОННЫЕ СИСТЕМЫ»

Разработка имитационной программы с использованием взаимодействующих процессов

Москва 2026

АННОТАЦИЯ

Документ содержит основные требования, которым должно соответствовать выполняемое индивидуальное задание. Задача должна быть реализована как консольное приложение для ОС Linux с использованием взаимодействующих процессов для имитации протекающих событий. Недетерминированные действия при необходимости можно реализовать с применением генераторов случайных чисел. Согласованное взаимодействие процессов необходимо организоваться с применением синхропримитивов.

Для каждого варианта приведен исходный предметный сценарий, вариативные входные параметры, обязательные наблюдаемые события, инварианты и условия завершения. Указанные сущности являются самостоятельными программными конструкциями отображаемыми на типы данных и функции. Необходимо провести их анализ на возможность реализации в качестве процессов.

Параметры, включая число процессов, их зависимость или независимость, определяются исходя из требований к заданию. В качестве бонусных возможностей могут использоваться аргументы командной строки, конфигурационный файл, журнал результатов, воспроизводимое начальное значение генератора случайных чисел, регулируемая задержка отображения и итоговая статистика.

СПИСОК СОКРАЩЕНИЙ

- API — Application Programming Interface. Интерфейс Программирования приложения — программный интерфейс, то есть описание способов взаимодействия одной компьютерной программы с другими.
- bash — Bourne Again shell. Одна из наиболее популярных командной оболочки UNIX. Популярна в Linux. Часто используется в качестве предустановленной командной оболочки.
- CLI — Command Line Interface. Интерфейс командной строки
- IDE — Integrated Development Environment. Интегрированная среда разработки — система программных средств, используемая программистами для разработки программного обеспечения.
- ISA — Ibstruction Set Architecture
- POSIX — Portable Operating System Interface. Переносимый интерфейс операционных систем — набор стандартов, описывающих интерфейсы между операционной системой и прикладной программой (системный API), библиотеку языка C и набор приложений и их интерфейсов.
- ОС — операционная система
- ЯП — язык программирования
- ЯПВУ — язык программирования высокого уровня

Оглавление

Минимальные требования к выполнению задания 5
Требования на оценку 4-5 баллов 6
Требования на оценку 6 баллов 7
Требования на оценку 7-8 баллов 7
Требования на оценку выше 8 баллов 8
Дополнительные требования 8
Оценка за выполненное задание 9

Минимальные требования к выполнению задания

Третье задание заключается в разработке программы, состоящей из взаимодействующих процессов. При этом используются знания и навыки, полученные в ходе изучения основ программирования, языка программирования C и разделов данной дисциплины. Необходимо обратить внимание на формирование консольного интерфейса программы, ее общей структуры, используя по возможности решения полученнные в ходе выполнения предыдущих заданий.

Так как большинство задач связаны с моделированием различных ситуаций, необходимо обеспечить **наглядное представление результатов**. Для интерактивного и наглядного отображения на дисплее рекомендуется использовать **временные задержки**.

В целом принятые решения определяется условием задачи и фантазией разработчика.

В ходе выполнения задания необходимо обеспечить следующие требования, позволяющие получить минимальную положительную оценку.

1. Предоставить отчет, в котором отразить следующую информацию:
   - привести фамилию, имя, отчество исполнителя, группу;
   - указать номер варианта и условие задачи;
   - представить сценарий, поясняющий, каким образом исходные сущности и их поведение отображаются в потоки и их взаимодействие;
   - описать реализацию пользовательского интерфейса;
   - описать варианты запуска программы и ее завершения в зависимости от используемых исходных данных.
2. Программа должна корректно выполняться в соответствии с условием задания.
3. Программа должна быть реализована на языке программирования C с использованием библиотек из libc, системных вызовов.
4. Вывод программы должен быть информативным, отражая все ключевые протекающие в ней события в терминах моделируемой пред-

метной области. Наблюдатель на основе вывода программы должен понимать, что в ней происходит в каждый момент времени ее работы.

5. Независимо от способа запуска программа должна завершаться в соответствии с возможными вариантами данных.

6. По возможности процессами должны быть представлены все моделируемые сущности, обладающие поведением или их семейства. Если по мнению разработчика такое решение невозможно, то число процессов и их ролевое распределение осуществляется по согласованию с преподавателем.

7. Процессы должны асинхронно взаимодействовать между собой, что необходимо поддерживать использованием изученных средств взаимодействия. Их использование определяет минимальные требования к организации программы.

8. Результаты работы программы должны быть отражены в отчете.

Функциональные требования на оценку в 4-5 баллов

Разработать консольное приложение, в котором единый родительский процесс запускает требуемое число дочерних процессов.

Требования:

1. Представить сценарий решаемой задачи поясняющий, каким образом исходные сущности и их поведение отображаются в процессы и их взаимодействие.

2. В зависимости от выбранной схемы решения программа может использовать обмен данными через общую или разделяемую память. Применение семафоров в программе возможно по одному из следующих вариантов:

- Множество процессов взаимодействуют с использованием именованных POSIX семафоров. Обмен данными ведется через разделяемую память в стандарте POSIX.
- Множество процессов взаимодействуют с использованием неименованных POSIX семафоров расположенных в разделяемой памяти. Обмен данными также ведется через разделяемую память в стандарте POSIX.

3. Реализовать завершение программы в соответствии с условием задачи, а также предусмотреть корректное завершение по прерыванию с клавиатуры по соответствующему сигналу.

4. В программе предусмотреть удаление семафоров и разделяемой памяти по ее завершению любым из способов.

Функциональные требования на оценку в 6 баллов

В дополнение к программе, разработанную на предыдущую оценку, необходимо разработать программу, в которой для взаимодействия процессов используется оставшийся вариант из двух, описанных выше, но не реализованный на предшествующую оценку.

Функциональные требования на оценку в 7-8 баллов

В дополнение к программам на предыдущие оценки необходимо разработать еще одну программу, состоящую из отдельных программ–процессов, запускаемых независимо друг от друга. С использованием семафоров и разделяемой памяти организовать между ними взаимодействие.

Требования:

1. Для завершения приложения использовать те же подходы, что и в предыдущих решениях.

2. Множество независимых процессов взаимодействуют с использованием именованных POSIX семафоров. Обмен данными ведется через разделяемую память в стандарте POSIX.

3. В отчет расширить информацией о добавленной реализации и привести соответствующие результаты работы программы.

Функциональные требования на оценку выше 8 баллов

В дополнение к программам на предыдущие оценки необходимо разработать приложение, состоящую из отдельных программ–процессов, запускаемых независимо друг от друга. С использованием семафоров и каналов (или очередей сообщений) организовать между ними взаимодействие.

Требования:

1. Для завершения приложения использовать те же подходы, что и в предыдущих решениях.

2. Множество независимых процессов взаимодействуют с использованием именованных POSIX семафоров. Обмен данными ведется через каналы или очереди сообщений в стандарте POSIX.

3. Отчет расширить информацией о добавленной реализации и привести соответствующие результаты работы программы.

Дополнительные требования

К дополнительным требованиям, влияющим на повышение оценки, относятся следующие.

1. В программе присутствуют комментарии, поясняющие выполняемые действия и описание используемых объектов и переменных.

2. Реализация ввода–вывода с применением системных вызовов и функций библиотеки libc, использующих дескрипторы файлов.

3. Независимо от способа запуска программа должна завершаться с использованием как по завершению работы, так и по ее прерыванию. Использование сигналов и изменение с их помощью условий прерывания программы повышает оценку.

4. Сохранение лога по завершении работы, к котором отражаются результаты выполнения, например, собранная статистика.

5. Формирование тестовых наборов данных и скриптов, обеспечивающие их запуск.

6. Предоставление при использовании ИИ всех промтов, приведших к формированию программного кода.

7. Критическая оценка программы, реализованной с использованием ИИ. Фиксация в отчете правок кода, если таковые были.

8. Реализация непредусмотренных в этом списке решений, которые могут впечатлить преподавателя и сподвигнуть его на повышение оценки.

Дополнительные требования являются необязательными, но отсутствие тех или иных функций в программе влияет на окончательную оценку.

Оценка за выполненное задание

Оценка за выполненное задание выставляется по 10-бальной шкале. В соответствии с ПОПАТКУС преподаватель имеет право не объяснять студентам, почему он поставил 8, а не 9 или 10. Оценки 9 и 10 могут выставляться исключительно по субъективному мнению преподавателя. Оценки 0-8 должны быть обоснованными. Максимальной оценкой является 8. Подход к выставлению оценки представлен на рисунке.

Оценка	Комментарий	На примерах Было дано задание: сделать космический корабль, готовый к запуску	Аналог в 5-балльной системе	Шкала в шакалах
10	Существенно превосходит ожидания, студент сделал значительно больше, чем от него требовал руководитель	Космический корабль уже долетел до точки назначения	5++	
9	Превосходит ожидания, студент сделал значительно больше, чем от него требовал руководитель	Космический корабль уже выходит на орбиту	5+	
8	Студент сделал все, что от него требовалось в полном объеме	Космический корабль готов к запуску	5	
7	Студент сделал почти все, что от него требовалось	Космический корабль к запуску готов, но еще не покрашен	4+	
6	В работе студента есть недочеты и/или задание сделано не полностью	В космическом корабле нет пульта управления	4	
5	Сделано мало либо в работе студента есть значительные недочеты и/или ошибки	Заказчику представлен только двигатель космического корабля	3+	
4	Сделано очень мало либо в работе студента есть очень значительные недочеты и/или ошибки	Заказчику представлена схема корабля и несколько винтиков	3	
1-3	Студент сделал критически недостаточно либо сделал не то, что требовалось	Сделали катамаран	Неуд.	
0	Выставляется, если студент не сдал документацию и/или недобросовестно работал над проектом, по мнению руководителя	Нет даже схемы космического корабля	Неуд.	